

Shared Mathematical Foundations:

Neural Networks, U-Net, and Multi-Modal Segmentation Architectures

TDA-SegUNet Thesis Project
Pontificia Universidad Javeriana

April 2026

Abstract

This document formalizes the mathematical framework underlying neural network architectures for image segmentation. Beginning from first principles (linear maps, compositions, gradient descent), we build toward a rigorous description of the U-Net architecture and its multi-modal extensions (InputFusion, BottleneckFusion). Each definition is accompanied by both intuitive analogies and concrete numerical examples in 2D and 3D. The gradient separation principle is formalized as a theorem in the multi-task learning framework. This document serves as the shared mathematical language referenced by both the topology paper and the geometry paper of the thesis.

1 Neural Networks as Compositions of Maps

1.1 The Building Blocks

Definition 1.1 (Affine Map). *An affine map is a function $A_{W,b} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ defined by*

$$A_{W,b}(x) = Wx + b \quad (1)$$

where $W \in \mathbb{R}^{m \times n}$ is the weight matrix and $b \in \mathbb{R}^m$ is the bias vector.

Example 1.2 (2D $\rightarrow$ 2D Affine Map). *Let $W = \begin{pmatrix} 2 & -1 \\ 0 & 3 \end{pmatrix}$, $b = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$. Then:*

$$A \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 & -1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 6 \end{pmatrix} + \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 5 \end{pmatrix}$$

The matrix W stretches and shears space; the bias b translates it.

Remark 1.3 (Analogy: The Assembly Line). *Think of an affine map as one station in an assembly line. The input x arrives (raw material), W reshapes it (cutting, bending), and b shifts it (repositioning). A neural network is the entire assembly line: many stations in sequence.*

Definition 1.4 (Activation Function). *An activation function is a non-linear map $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ applied element-wise. The Rectified Linear Unit (ReLU) is:*

$$\sigma(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \quad (2)$$

Example 1.5 (ReLU on a Vector). $\sigma \begin{pmatrix} 1 \\ 5 \end{pmatrix} = \begin{pmatrix} \max(0, 1) \\ \max(0, 5) \end{pmatrix} = \begin{pmatrix} 1 \\ 5 \end{pmatrix}$. But $\sigma \begin{pmatrix} -3 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$. ReLU “kills” negative values. Without it, composing affine maps yields another affine map (linear algebra), and the network cannot learn non-linear functions.

Definition 1.6 (Feed-Forward Neural Network). *A feed-forward neural network of depth L is a composition:*

$$f_\theta = A_L \circ \sigma \circ A_{L-1} \circ \sigma \circ \dots \circ \sigma \circ A_1 \quad (3)$$

where $A_\ell = A_{W_\ell, b_\ell}$ are affine maps and $\theta = \{W_1, b_1, \dots, W_L, b_L\}$ is the parameter set.

Example 1.7 (A 2-Layer Network: Step by Step). *Let $f_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^1$ with $W_1 = \begin{pmatrix} 1 & -1 \\ 2 & 0 \end{pmatrix}$, $b_1 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$, $W_2 = \begin{pmatrix} 1 & 1 \end{pmatrix}$, $b_2 = (0)$.*

For input $x = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$:

Layer 1 (affine): $z_1 = W_1 x + b_1 = \begin{pmatrix} 2 \\ 5 \end{pmatrix}$

Layer 1 (activate): $h_1 = \sigma(z_1) = \begin{pmatrix} 2 \\ 5 \end{pmatrix}$ (both positive)

Layer 2 (affine): $f_\theta(x) = W_2 h_1 + b_2 = 7$

Now for $x' = \begin{pmatrix} 0 \\ 3 \end{pmatrix}$: $z_1 = \begin{pmatrix} -3 \\ -1 \end{pmatrix}$, $h_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, $f_\theta(x') = 0$.

ReLU killed both negative pre-activations, collapsing the output to zero. This is how the network creates piecewise-linear decision boundaries.

1.2 Universal Approximation

Theorem 1.8 (Universal Approximation, Cybenko 1989; Hornik 1991). *For any continuous function $g : K \rightarrow \mathbb{R}$ on a compact set $K \subset \mathbb{R}^n$ and any $\varepsilon > 0$, there exists a neural network f_θ with one hidden layer of sufficient width such that $\|f_\theta - g\|_\infty < \varepsilon$.*

Remark 1.9. *This theorem says neural networks CAN approximate anything. It does NOT say they will LEARN to — that depends on the optimization algorithm and training data. The gap between approximation theory and learning practice is exactly where architecture design matters.*

2 Convolutional Operations

2.1 Discrete Convolution on Images

Definition 2.1 (2D Discrete Convolution). *Given an image $I \in \mathbb{R}^{H \times W}$ and a kernel $K \in \mathbb{R}^{k \times k}$, the convolution is:*

$$(I * K)(i, j) = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} K(p, q) \cdot I(i + p, j + q) \quad (4)$$

This is a local operation: the output at position (i, j) depends only on a $k \times k$ neighborhood of (i, j) in the input.

Example 2.2 (3×3 Edge Detection). *Let $I =$*

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \quad (\text{a } 4 \times 4 \text{ image with a } 2 \times 2 \text{ white square})$$

and the Laplacian kernel $K =$

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}.$$

*Computing $(I * K)(1, 1)$: center kernel at position $(1, 1)$:*

$$\begin{aligned} & 0(0) + (-1)(0) + 0(0) \\ & + (-1)(0) + 4(1) + (-1)(1) \\ & + 0(0) + (-1)(1) + 0(1) = 2 \end{aligned}$$

The non-zero output at $(1, 1)$ indicates an edge. Interior pixels (like $(1, 2)$ if the square were larger) would give 0 because the Laplacian detects curvature.

Definition 2.3 (Multi-Channel Convolution). *A convolutional layer maps C_{in} input channels to C_{out} output channels:*

$$Conv_{W,b}(X)_{c,i,j} = \sum_{c'=1}^{C_{in}} \sum_{p,q} W_{c,c',p,q} \cdot X_{c',i+p,j+q} + b_c \quad (5)$$

where $W \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$ is a 4D tensor. Each output channel c has its own set of C_{in} kernels summed together.

Remark 2.4 (Analogy: The Translator). *A single-channel convolution is like reading a page with a magnifying glass — you see one local patch at a time. Multi-channel convolution is like having C_{out} translators, each looking at the same page through C_{in} colored filters. Each translator produces a different “interpretation” (feature map) of the same input.*

Example 2.5 (Parameter Count). *A Conv layer with $C_{in} = 3$, $C_{out} = 32$, $k = 3$: parameters = $32 \times 3 \times 3 \times 3 + 32 = 896$. This is the first layer of InputFusion, processing the 3-channel input (image + PI_0 + PI_1).*

2.2 Pooling and Spatial Reduction

Definition 2.6 (Max Pooling). *Max pooling with window $s \times s$ and stride s :*

$$Pool(X)_{c,i,j} = \max_{0 \leq p,q < s} X_{c,si+p,sj+q} \quad (6)$$

This reduces spatial dimensions by factor s while preserving the strongest activation in each local window.

Example 2.7 (2×2 Max Pooling).

$$Pool \begin{pmatrix} 1 & 3 & 0 & 2 \\ 5 & 2 & 1 & 0 \\ 0 & 1 & 4 & 3 \\ 2 & 0 & 1 & 6 \end{pmatrix} = \begin{pmatrix} 5 & 2 \\ 2 & 6 \end{pmatrix}$$

The 4×4 input becomes 2×2 . Spatial detail is lost, but the dominant feature in each 2×2 block is retained.

3 The U-Net Architecture

3.1 Formal Definition

Definition 3.1 (Encoder Block). *An encoder block at level ℓ is:*

$$E_\ell = Pool_{2 \times 2} \circ \sigma \circ BN \circ Conv_2 \circ \sigma \circ BN \circ Conv_1 \quad (7)$$

mapping $\mathbb{R}^{C_\ell \times H_\ell \times W_\ell} \rightarrow \mathbb{R}^{2C_\ell \times H_\ell/2 \times W_\ell/2}$.

The pre-pooling features (before Pool) are stored as the skip connection:

$$s_\ell = (\sigma \circ BN \circ Conv_2 \circ \sigma \circ BN \circ Conv_1)(X_\ell) \in \mathbb{R}^{2C_\ell \times H_\ell \times W_\ell} \quad (8)$$

Definition 3.2 (Decoder Block). *A decoder block at level ℓ is:*

$$D_\ell = \sigma \circ BN \circ Conv_2 \circ \sigma \circ BN \circ Conv_1 \circ Cat(\cdot, s_\ell) \circ Up \quad (9)$$

where Up is $2 \times$ bilinear upsampling and $Cat(\cdot, s_\ell)$ concatenates along the channel dimension with the skip connection from the corresponding encoder level.

Definition 3.3 (U-Net). *A U-Net of depth L with initial channels C_0 is:*

$$f_\theta = \sigma_{out} \circ Conv_{1 \times 1} \circ D_1 \circ D_2 \circ \dots \circ D_L \circ B \circ E_L \circ \dots \circ E_1 \quad (10)$$

where B is the bottleneck (two convolutions without pooling), σ_{out} is sigmoid, and $Conv_{1 \times 1}$ maps from C_0 channels to 1.

Remark 3.4 (Analogy: Compression and Reconstruction). *The encoder is like writing a summary of a book: each level compresses the text (fewer pages = lower resolution) while highlighting key themes (more channels = richer features). The bottleneck is the most compressed summary. The decoder reconstructs the full book from the summary, using the skip connections as “margin notes” that preserve details lost during summarization.*

3.2 Numerical Walkthrough: Tensor Shapes

Example 3.5 (Our U-Net, 128×128 Input). With $L = 3$, $C_0 = 1$ (Baseline):

Stage	Shape	Params
Input	$1 \times 128 \times 128$	—
E_1 skip	$32 \times 128 \times 128$	—
E_1 out	$32 \times 64 \times 64$	9,280
E_2 skip	$64 \times 64 \times 64$	—
E_2 out	$64 \times 32 \times 32$	55,552
E_3 skip	$128 \times 32 \times 32$	—
E_3 out	$128 \times 16 \times 16$	221,440
B	$256 \times 8 \times 8$	885,248
D_3	$128 \times 16 \times 16$	663,808
D_2	$64 \times 32 \times 32$	55,488
D_1	$32 \times 64 \times 64$	13,920
Output	$1 \times 128 \times 128$	33
Total		1,926,433

Note: 46% of parameters are in the bottleneck (B). This is the “information bottleneck” where the most abstract representation lives.

3.3 Skip Connections and Gradient Flow

Proposition 3.6 (Gradient Highway). The gradient of the loss $\mathcal{L}$ with respect to encoder features at level ℓ has two paths:

$$\frac{\partial \mathcal{L}}{\partial e_\ell} = \underbrace{\frac{\partial \mathcal{L}}{\partial e_\ell} \Big|_{\text{deep path}}}_{\text{through all deeper layers}} + \underbrace{\frac{\partial \mathcal{L}}{\partial e_\ell} \Big|_{\text{skip path}}}_{\text{direct via Cat}} \quad (11)$$

The skip path is “short” (passes through only the decoder level ℓ and the output layer), preventing vanishing gradients.

Proof sketch. Since D_ℓ receives e_ℓ via concatenation: $D_\ell = g(\text{Cat}(\text{Up}(d_{\ell+1}), e_\ell))$, the partial derivative $\partial D_\ell / \partial e_\ell$ includes the identity map on the concatenated channels (the skip simply copies e_ℓ into the decoder). The gradient through this path involves only the Jacobian of the convolutions in D_ℓ and the output layer, avoiding the product of Jacobians through the entire bottleneck. $\square$

Remark 3.7 (Analogy: Two Phone Lines). Without skip connections, a message from the encoder to the output must pass through every intermediate person (like a game of telephone) — the message degrades. With skip connections, there’s a direct line from each encoder level to its decoder counterpart. The decoder hears both the “degraded” deep message (semantic meaning) and the “clear” direct message (spatial detail).

4 Multi-Modal Architectures

4.1 The Multi-Modal Segmentation Problem

Our segmentation task uses two modalities: the intensity image $I \in \mathbb{R}^{1 \times H \times W}$ and topological features $\tau = (PI_0, PI_1) \in \mathbb{R}^{2 \times H \times W}$. Three architectural strategies exist for combining them.

4.2 Baseline: Single-Modal Reference

Definition 4.1 (BaselineUNet).

$$f_{BL}(I) = \text{UNet}(I), \quad I \in \mathbb{R}^{1 \times H \times W} \quad (12)$$

Standard U-Net ignoring τ . Parameters: $|\theta_{BL}| = 1,926,433$.

4.3 InputFusion: Early Concatenation

Definition 4.2 (InputFusion).

$$f_{IF}(I, \tau) = \text{UNet}(\text{Cat}(I, PI_0, PI_1)) \quad (13)$$

where $\text{Cat} : \mathbb{R}^{1 \times H \times W} \times \mathbb{R}^{2 \times H \times W} \rightarrow \mathbb{R}^{3 \times H \times W}$ is channel concatenation. A single encoder with $C_0 = 3$ processes all modalities. Parameters: $|\theta_{IF}| = 1,927,009$.

Example 4.3 (First Conv in InputFusion). The first Conv layer has kernel $W_1 \in \mathbb{R}^{32 \times 3 \times 3 \times 3}$. For output channel c at position (i, j) :

$$\begin{aligned} \text{out}_{c,i,j} = & \sum_{p,q} W_{c,0,p,q} \cdot I(i+p, j+q) \\ & + \sum_{p,q} W_{c,1,p,q} \cdot PI_0(i+p, j+q) \\ & + \sum_{p,q} W_{c,2,p,q} \cdot PI_1(i+p, j+q) + b_c \end{aligned} \quad (14)$$

Intensity and topology interact from the very first layer. The same 32 output channels must encode both spatial and topological information.

4.4 BottleneckFusion: Dual Encoder

Definition 4.4 (BottleneckFusion).

$$f_{BF}(I, \tau) = \text{Dec}(\text{Fuse}(E_{sp}(I), E_{tp}(\tau))) \quad (15)$$

with two independent encoders:

$$E_{sp} : \mathbb{R}^{1 \times H \times W} \rightarrow \mathbb{R}^{256 \times \frac{H}{8} \times \frac{W}{8}} \quad (\text{spatial}) \quad (16)$$

$$E_{tp} : \mathbb{R}^{2 \times H \times W} \rightarrow \mathbb{R}^{128 \times \frac{H}{8} \times \frac{W}{8}} \quad (\text{topological}) \quad (17)$$

Fusion at the bottleneck:

$$\text{Fuse}(s, t) = \text{Conv}_{1 \times 1}(\text{Cat}(s, t)) : \mathbb{R}^{384} \rightarrow \mathbb{R}^{256} \quad (18)$$

Key property: $\theta_{sp} \cap \theta_{tp} = \emptyset$ (disjoint parameter sets). Skip connections come from E_{sp} only. Parameters: $|\theta_{BF}| = 2,319,361$.

Example 4.5 (Fusion as Pointwise Linear Map). At spatial position (i, j) in the bottleneck (8×8) , the fusion takes vector $v = \text{Cat}(s_{i,j}, t_{i,j}) \in \mathbb{R}^{384}$ and maps it to $\mathbb{R}^{256}$ via:

$$\text{Fuse}(v) = W_f v + b_f, \quad W_f \in \mathbb{R}^{256 \times 384} \quad (19)$$

This is a pointwise linear map — the same matrix W_f is applied independently at every spatial position. It learns which combinations of spatial features (s) and topological features (t) are relevant for segmentation.

Remark 4.6 (Analogy: Two Experts). *InputFusion* is like one analyst reading a report that mixes financial data and market sentiment on the same page — they must context-switch constantly. *BottleneckFusion* is like two specialists: a financial analyst and a market psychologist. Each reads their own data independently, then they meet in a boardroom (the bottleneck) to combine their insights into a single recommendation.

4.5 Architecture Comparison

Table 1: Architectural comparison.

	BL	IF	BF
Input channels	1	3	1+2
Encoders	1	1	2
Params	1.93M	1.93M	2.32M
Fusion point	—	Input	Bottleneck
Skip source	Single	Single	Spatial only
θ overlap	—	Full	$\emptyset$

5 Loss Functions and Multi-Task Learning

5.1 SoftDiceLoss

Definition 5.1 (Soft Dice Loss).

$$\mathcal{L}_{\text{Dice}}(P, G) = 1 - \frac{2 \sum_i P_i G_i + \varepsilon}{\sum_i P_i + \sum_i G_i + \varepsilon} \quad (20)$$

where $P_i \in [0, 1]$ is the predicted probability at pixel i , $G_i \in \{0, 1\}$ is the ground truth, and $\varepsilon = 10^{-5}$ prevents division by zero.

Example 5.2 (Dice Computation). $P = (0.9, 0.8, 0.1, 0.2)$, $G = (1, 1, 0, 0)$.

$$\begin{aligned} \sum P_i G_i &= 0.9 + 0.8 = 1.7 \\ \sum P_i &= 2.0, \quad \sum G_i = 2.0 \\ \text{Dice} &= \frac{2(1.7)}{4.0} = 0.85, \quad \mathcal{L}_{\text{Dice}} = 0.15 \end{aligned}$$

5.2 Betti Matching Loss

Definition 5.3 (Betti Matching Loss, Stucki et al. 2023). Given prediction P and ground truth G , define $C = \max(P, G)$. The inclusions $P \hookrightarrow C$ and $G \hookrightarrow C$ induce matchings on persistent homology. The loss penalizes unmatched features:

$$\mathcal{L}_{\text{BM}}(P, G) = \sum_{\substack{(b,d) \in \text{Dgm}(P) \\ \text{unmatched}}} (d-b)^2 + \sum_{\substack{(b',d') \in \text{Dgm}(G) \\ \text{unmatched}}} (d'-b')^2 \quad (21)$$

5.3 The Gradient Conflict Problem

Definition 5.4 (Multi-Task Loss). The combined training loss is:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_{\text{Dice}} + \lambda \mathcal{L}_{\text{BM}} \quad (22)$$

with curriculum schedule: $\lambda = 0$ (warmup), $\lambda \in [0, 0.3]$ (ramp), $\lambda = 0.3$ (full).

Definition 5.5 (Gradient Conflict, Yu et al. 2020). Two task gradients $g_A = \nabla_{\theta} \mathcal{L}_{\text{Dice}}$ and $g_B = \nabla_{\theta} \mathcal{L}_{\text{BM}}$ are in conflict at parameter θ if:

$$\cos(g_A, g_B) = \frac{\langle g_A, g_B \rangle}{\|g_A\| \cdot \|g_B\|} < 0 \quad (23)$$

Definition 5.6 (The Tragic Triad, Yu et al. 2020). Negative transfer in multi-task learning occurs when three conditions coincide:

1. Conflicting gradients: $\cos(g_A, g_B) < 0$
2. High positive curvature along the conflicting direction
3. Magnitude imbalance: $\|g_A\| \gg \|g_B\|$

6 The Gradient Separation Principle

Proposition 6.1 (Gradient Conflict in InputFusion). Let p_c be a critical pixel that is a false positive in the Dice sense ($P(p_c) > 0.5$, $G(p_c) = 0$) but topologically necessary (closing a gap that restores β_1). Then for any shared encoder parameter w in *InputFusion*:

$$\text{sign} \left(\frac{\partial \mathcal{L}_{\text{Dice}}}{\partial w} \right) = -\text{sign} \left(\frac{\partial \mathcal{L}_{\text{BM}}}{\partial w} \right) \quad (24)$$

at the critical pixel, creating gradient conflict.

Proof sketch. The Dice gradient pushes $P(p_c) \rightarrow G(p_c) = 0$ (reducing false positive). The Betti gradient pushes $P(p_c) \rightarrow 1$ (closing the topological gap). By the chain rule, these opposing pixel-level signals propagate to opposing parameter-level signals through any shared computational path. $\square$

Theorem 6.2 (Gradient Separation in BottleneckFusion). *In BottleneckFusion, the effective gradient conflict is structurally reduced:*

$$\cos(g_A^{sp}, g_B^{sp}) \approx 0 \quad (\text{spatial encoder}) \quad (25)$$

$$\cos(g_A^{tp}, g_B^{tp}) > 0 \quad (\text{topological encoder}) \quad (26)$$

where $g_A^{sp} = \nabla_{\theta_{sp}} \mathcal{L}_{Dice}$ and $g_B^{sp} = \nabla_{\theta_{sp}} \mathcal{L}_{BM}$, etc.

Proof sketch. (1) The spatial encoder E_{sp} processes only intensity I . The Betti loss gradient $\nabla_{\theta_{sp}} \mathcal{L}_{BM}$ must propagate through the fusion layer, the bottleneck, and the entire spatial encoder. This path *dilutes* the topological signal across all spatial parameters — most spatial parameters affect many pixels, not specifically critical ones. Thus $\|g_B^{sp}\|$ is small relative to $\|g_A^{sp}\|$, and their cosine similarity is near zero (approximately orthogonal).

(2) The topological encoder E_{tp} processes persistence images, which already encode topological information. Both the Dice gradient (which benefits from good topological features indirectly) and the Betti gradient (which benefits directly) push E_{tp} toward better topological representations. Their gradients are aligned.

Since neither encoder satisfies condition (1) of the Tragic Triad (Definition ??), negative transfer is avoided. $\square$

Corollary 6.3. *BottleneckFusion is an architectural solution to gradient conflict. It achieves structurally what PCGrad [?] achieves algorithmically (gradient projection), but without runtime overhead.*

6.1 Empirical Verification

Table 2: Evidence for gradient separation.

Evidence	IF	BF
β_1 (dice+beti)	0.010	0.000
β_1 (beti only)	0.020	0.005
Dice (beti only)	0.966	0.968

Under pure Betti loss (no Dice supervision), BF maintains *higher* Dice than IF (0.968 vs 0.966). This is counterintuitive: without Dice supervision, how does BF preserve pixel quality? Because the spatial encoder, receiving weak Betti gradients (near-zero cosine similarity), retains its learned spatial representations without topological corruption.

7 Evaluation Metrics

Definition 7.1 (Betti Number Error).

$$\beta_k^{err} = |\beta_k(P_{binary}) - \beta_k(G)| \quad (27)$$

where β_k is the k -th Betti number computed on the binarized prediction ($P > 0.5$) and ground truth.

Definition 7.2 (Hausdorff Distance 95%).

$$HD_{95}(P, G) = \text{percentile}_{95} \left(\max \left(\sup_{p \in \partial P} d(p, \partial G), \sup_{g \in \partial G} d(g, \partial P) \right) \right) \quad (28)$$

This measures the 95th percentile of boundary distances, robust to outliers.

Remark 7.3. *Dice measures volume overlap, β_k measures structural correctness, HD_{95} measures boundary precision. A complete evaluation requires all three: a segmentation can have perfect Dice with wrong topology, or perfect topology with poor boundary position.*

References

- [1] Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*.
- [2] Etmann, C. et al. (2023). A Unified Framework for U-Net Design and Analysis. arXiv:2305.19638.
- [3] Zhou, D.-X. (2020). Universality of deep convolutional neural networks. *ACHA*, 48(2), 787–794.
- [4] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *MCSS*, 2, 303–314.
- [5] Stucki, N. et al. (2023). Topologically Faithful Image Segmentation via Induced Matching. *ICML*.
- [6] Yu, T. et al. (2020). Gradient Surgery for Multi-Task Learning. *NeurIPS*.
- [7] Vandenhende, S. et al. (2021). Multi-Task Learning for Dense Prediction Tasks: A Survey. *IEEE TPAMI*.
- [8] Adams, H. et al. (2017). Persistence Images: A Stable Vector Representation. *JMLR*, 18(8), 1–35.
- [9] He, K. et al. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
- [10] Berger, C. et al. (2025). Pitfalls of Topology-Aware Image Segmentation. *IPMI*.